

Ofren Collect — Requirements Specification

Project: Ofren Collect — Recurring billing + automatic reconciliation on Monnify **Author:** Ofren Technologies **Context:** APIConf Lagos 2026 x Monnify Developer Challenge **Status:** Draft for hackathon build — updated for multi-tenant, secure, audited architecture **Last updated:** July 2026

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for Ofren Collect, a system that lets small Nigerian SaaS and service businesses collect recurring payments and automatically reconcile inflows to the correct customer and invoice, using the Monnify API.

It is written to guide a time-boxed hackathon build. Requirements are prioritised so the team can ship a complete, demonstrable core slice first and layer enhancements only if time allows.

1.2 Scope

Ofren Collect covers: multi-tenant sign-up and authentication (each business sees only its own data), defining billing plans, enrolling customers onto plans, provisioning a dedicated Monnify reserved account per subscription, detecting and verifying inflows via webhooks, reconciling payments against invoices, surfacing payment status on a real-time dashboard, audit logging of every API call, rate limiting, resilient (retry + circuit breaker) calls to Monnify, and (as stretches) settling collected funds to a merchant account and an optional AI natural-language query over collections.

The build targets a **single backend instance** using **PostgreSQL** (host-agnostic — local container or any managed Postgres) and only **free / open-source** components.

Out of scope for this build: multi-currency beyond NGN, live/production payments, horizontal scale-out across multiple instances (and the Redis backplane / distributed cache it would require), external identity providers / SSO / MFA, tax handling, dunning email campaigns, and mobile apps.

1.3 Definitions

Term	Meaning
Plan	A named recurring charge (amount + interval), e.g. "Basic — ₦5,000/month"
Customer	An individual or business being billed
Subscription	A customer enrolled on a plan, with a dedicated reserved account
Reserved account	A Monnify virtual account uniquely tied to one subscription
Invoice	The amount due for one billing period of a subscription
Reconciliation	Matching an inflow to the owning subscription and invoice, then setting status
Inflow	Money paid into a reserved account
Tenant	One business using the system; the isolation boundary for all data
User	A login belonging to exactly one tenant (role: Owner or Staff)
Audit entry	A recorded API call: who, what, when, request/response, status
Circuit breaker	A resilience pattern that stops calling a failing dependency to fail fast
WAT	West Africa Time (UTC+1)

1.4 Requirement priority

Priority uses MoSCoW: **Must** (core, required for a valid submission), **Should** (strong value if time allows), **Could** (stretch), **Won't** (explicitly excluded this build).

2. Functional Requirements

2.0 Authentication & multi-tenancy

ID	Requirement	Priority
FR-0.1	The system shall allow a business to self-register, creating a tenant and an owner user.	Must
FR-0.2	The system shall authenticate users by email and password and issue a signed token (JWT) carrying the user's tenant and role.	Must
FR-0.3	The system shall store passwords only as salted hashes, never in plain text.	Must
FR-0.4	The system shall require a valid token for all business endpoints and the real-time channel; only registration, login, and the Monnify webhook are exempt.	Must
FR-0.5	The system shall derive the acting tenant from the authenticated token, never from client-supplied input.	Must
FR-0.6	The system shall isolate all tenant-owned data so a user can read or change only their own tenant's records.	Must
FR-0.7	The system shall scope real-time notifications per tenant, so one tenant never receives another's events.	Must
FR-0.8	The system shall resolve the tenant for an incoming Monnify webhook from the reserved account paid into (the webhook carries no token).	Must
FR-0.9	The system shall restrict sensitive actions (e.g. viewing the audit log) to the Owner role.	Should

2.1 Plan management

ID	Requirement	Priority
FR-1.1	The system shall allow a user to create a plan with a name, amount (NGN), and billing interval (e.g. monthly).	Must
FR-1.2	The system shall list all existing plans.	Must
FR-1.3	The system shall validate that plan amount is greater than zero and name is non-empty.	Must
FR-1.4	The system shall allow a plan to be deactivated so no new subscriptions can use it.	Could

2.2 Customer & subscription management

ID	Requirement	Priority
FR-2.1	The system shall allow a user to register a customer with a name and email.	Must
FR-2.2	The system shall allow a customer to be enrolled onto a plan, creating a subscription.	Must
FR-2.3	On enrolment, the system shall request a dedicated Monnify reserved account for that subscription and persist the returned account number and bank details.	Must
FR-2.4	The system shall generate a unique internal account reference per subscription and send it to Monnify as the account reference.	Must
FR-2.5	The system shall create the first invoice for a subscription on enrolment, with a due date derived from the billing interval.	Must
FR-2.6	The system shall display each subscription's reserved account number so it can be shared with the customer for payment.	Must
FR-2.7	The system shall allow a subscription to be cancelled, stopping further invoicing.	Should

2.3 Payment detection & verification

ID	Requirement	Priority
FR-3.1	The system shall expose a webhook endpoint to receive Monnify transaction notifications.	Must
FR-3.2	The system shall acknowledge webhook receipt with an HTTP 200 promptly, before or independently of downstream processing.	Must
FR-3.3	The system shall verify the authenticity of each incoming webhook (signature/hash check) before trusting its contents.	Must
FR-3.4	The system shall independently verify each transaction with Monnify's transaction verification endpoint before reconciling.	Must
FR-3.5	The system shall record each processed transaction with its Monnify reference, amount, paid-into account, and timestamp.	Must
FR-3.6	The system shall ignore/deduplicate a transaction reference it has already processed (idempotency).	Must

2.4 Reconciliation

ID	Requirement	Priority
FR-4.1	The system shall resolve an inflow to its owning subscription using the reserved account it was paid into.	Must
FR-4.2	The system shall apply the inflow amount to the subscription's current open invoice.	Must
FR-4.3	The system shall mark an invoice Paid when the paid amount equals the amount due.	Must
FR-4.4	The system shall mark an invoice Underpaid when the paid amount is less than the amount due, and record the outstanding balance.	Must
FR-4.5	The system shall mark an invoice Overpaid when the paid amount exceeds the amount due, and record the credit.	Should
FR-4.6	On full payment, the system shall advance the subscription to the next billing period and create the next invoice.	Should
FR-4.7	The system shall record an inflow that matches no subscription as Unmatched and surface it for review rather than discarding it.	Should
FR-4.8	The system shall flag a subscription as Overdue when an invoice passes its due date without being fully paid.	Should

2.5 Dashboard & reporting

ID	Requirement	Priority
FR-5.1	The system shall present a dashboard listing the tenant's subscriptions with customer, plan, reserved account, next due date, and current status, updating in real time as payments are reconciled.	Must
FR-5.2	The system shall visually distinguish invoice/subscription statuses (e.g. paid, underpaid, overdue).	Should
FR-5.3	The system shall display summary figures: total collected in the current period, count overdue, count of unmatched payments.	Should
FR-5.4	The system shall provide forms to create plans and enrol customers from the dashboard.	Must

2.6 Audit logging

ID	Requirement	Priority
FR-8.1	The system shall record every API call as an audit entry capturing correlation id, user, tenant, HTTP method, path, response status, and timestamp.	Must
FR-8.2	The system shall record a sanitised view of request and response bodies, redacting secrets, tokens, passwords, and full card/account numbers.	Must
FR-8.3	The system shall allow an Owner to query their own tenant's audit trail; audit reads shall themselves be tenant-scoped.	Should
FR-8.4	The system shall emit a correlation id on responses and in logs so a request can be traced across the audit store and application logs.	Should

2.7 Settlement (stretch)

ID	Requirement	Priority
FR-6.1	The system shall allow collected funds to be disbursed to a merchant bank account via a Monnify transfer.	Could
FR-6.2	The system shall record each settlement transfer with its reference and status.	Could

2.8 AI assistant — natural-language query (stretch)

ID	Requirement	Priority
FR-7.1	The system shall answer a natural-language question about the tenant's own collections (e.g. "how much did I collect this week?") over real, tenant-scoped data.	Could
FR-7.2	If the AI feature is not reliable, it shall be excluded from the submission rather than shipped partially.	Must (governance)
FR-7.3	The AI assistant shall be read-only: it shall never create, modify, cancel, or move money, and shall not return figures not grounded in the tenant's actual data.	Must (if built)
FR-7.4	The AI assistant shall sit behind a provider-agnostic interface and a feature flag, so it can be disabled or its provider swapped by configuration alone.	Must (if built)
FR-7.5	The AI assistant shall respect tenant isolation, audit logging, and rate limiting exactly as other endpoints do.	Must (if built)

3. Non-Functional Requirements

3.1 Security

ID	Requirement	Priority
NFR-1.1	No secrets (API keys, secret keys, contract codes) shall be committed to the repository, in the working tree or git history.	Must
NFR-1.2	Secrets shall be supplied via environment variables or .NET user-secrets; the repo shall contain only a placeholder <code>.env.example</code> .	Must
NFR-1.3	The system shall use only Monnify sandbox credentials and endpoints; no live keys or live transactions.	Must
NFR-1.4	Webhook payloads shall be authenticated (signature/hash verification) before being acted upon.	Must
NFR-1.5	Transaction outcomes shall be confirmed via server-side verification, never trusted from the webhook body alone.	Must
NFR-1.6	The system shall not log full secret values or sensitive payloads at any log level.	Should
NFR-1.7	The system shall enforce tenant isolation at the data-access layer (a global query filter), so isolation does not depend on each query remembering to filter.	Must
NFR-1.8	The system shall sign tokens with a secret held only in configuration/secrets, never in the repository or the frontend.	Must
NFR-1.9	The system shall rate-limit requests — per tenant/user on authenticated routes and per IP on anonymous auth and webhook routes — returning 429 when exceeded.	Should

3.2 Reliability & correctness

ID	Requirement	Priority
NFR-2.1	Reconciliation shall be idempotent: re-delivery of the same transaction shall not double-count.	Must
NFR-2.2	The system shall handle Monnify API failures gracefully (surface an error, avoid corrupt state) rather than crashing.	Must
NFR-2.3	A failed webhook processing attempt shall not lose the notification; the inflow shall remain resolvable.	Should
NFR-2.4	The auth token shall be cached and refreshed before expiry rather than re-fetched per call.	Should
NFR-2.5	Outbound calls to Monnify shall be wrapped in retry-with-backoff, timeout, and a circuit breaker, so transient faults are retried and a sustained outage fails fast instead of cascading.	Should
NFR-2.6	Verified webhook notifications shall be persisted durably (inbox pattern) within the same transaction as the payment record, so a crash mid-processing cannot lose an acknowledged payment.	Should

3.3 Usability & clarity

ID	Requirement	Priority
NFR-3.1	A judge shall be able to clone and run the project locally using only the README, with no undocumented steps.	Must
NFR-3.2	The system shall seed demo data on first run so it is not empty when first opened.	Must
NFR-3.3	The README shall include a clear problem statement, setup guide, and architecture overview (clarity and storytelling).	Must
NFR-3.4	The dashboard shall be understandable without training — statuses and figures self-explanatory.	Should

3.4 Maintainability & portability

ID	Requirement	Priority
NFR-4.1	The system shall use PostgreSQL (free, host-agnostic), runnable locally via a Postgres container or against any managed Postgres provider, with schema created by EF Core migrations and seeded on first run.	Must
NFR-4.2	Monnify integration shall be isolated behind a service layer so endpoints/fields can be corrected in one place.	Should
NFR-4.3	The codebase shall run on the .NET 10 SDK (or .NET 8) across Windows, macOS, and Linux.	Should
NFR-4.4	Configuration shall be externalised (no hard-coded environment-specific values).	Should
NFR-4.5	External integrations (Monnify, AI provider) shall sit behind interfaces so implementations can be swapped or disabled by configuration.	Should

3.5 Performance

ID	Requirement	Priority
NFR-5.1	The webhook endpoint shall acknowledge within a few seconds to avoid Monnify retries.	Must
NFR-5.2	The dashboard shall load its list view responsively for a demo-scale dataset (tens to low hundreds of records).	Should

3.6 Delivery constraints

ID	Requirement	Priority
NFR-6.1	The submission shall include a public code repository with a step-by-step local setup guide.	Must
NFR-6.2	The submission shall contain no broken links or exposed secrets.	Must
NFR-6.3	The submission shall be completed by 12:00 WAT on 21 July 2026.	Must
NFR-6.4	The project shall be shared on social media with #APIConfXMonnify and #DeveloperChallenge.	Must
NFR-6.5	The team shall consist of exactly two members, both registered in-person attendees of API Conference Lagos 2026.	Must

4. Assumptions & dependencies

- Monnify sandbox provides reserved-account creation, transaction webhooks, transaction verification, and transfers; exact request/response shapes are to be confirmed against the live sandbox before coding.
- The team can expose a local webhook endpoint to Monnify during testing (e.g. via a tunnelling tool).
- A valid Monnify sandbox account with API key, secret key, and contract code is available.
- Both teammates are registered conference attendees and will attend in person.

5. Acceptance criteria (core demo)

The build is considered successful for submission when the following end-to-end flow works and is documented:

1. A business registers and logs in, receiving a token scoped to its tenant.
2. Within that tenant, a plan is created.
3. A customer is enrolled, and a Monnify reserved account is provisioned and shown.
4. A sandbox payment into that account is detected via webhook, verified, and resolved to the correct tenant.
5. The payment is reconciled and the invoice status updates correctly (paid / underpaid).
6. The dashboard reflects the updated status in real time, showing only that tenant's data.
7. The repository is public, secret-free, and runs from a clean clone using the README.

Stretch features (settlement, overpayment handling, AI assistant) are not required for acceptance and may be omitted if incomplete.

5.1 Phased delivery (given expanded scope)

Because this specification now includes auth, multi-tenancy, audit logging, rate limiting, and resilience alongside the core, the team builds in priority order so the most valuable, demonstrable slice exists first:

1. **Core reconciliation slice (P1):** plan → enrol → reserved account → webhook → verify → reconcile → dashboard.
Demonstrable on its own.
2. **Security layer:** authentication, then multi-tenancy (tenant stamping + global query filter + per-tenant real-time).
3. **Operational layer:** audit logging, rate limiting, retry/circuit breaker.
4. **Stretches:** settlement, then the AI assistant — built last, cut first.

If time runs short, cut from the bottom of this list; never let a lower tier compromise a working core.

6. Traceability summary

Judging criterion	Requirements that address it
Solves a real problem / useful Monnify workflow	FR-2.x, FR-3.x, FR-4.x
Clarity and storytelling	NFR-3.1, NFR-3.3, FR-5.x
Technical depth	FR-0.x (auth/tenancy), FR-3.3–3.6, FR-4.x, FR-8.x (audit), NFR-1.7–1.9, NFR-2.1, NFR-2.5–2.6
Practical, well-designed AI (not slop)	FR-7.1, FR-7.3–7.5
No errors / broken links / exposed secrets	NFR-1.1–1.4, NFR-1.8, NFR-6.2
Public repo with setup guide	NFR-3.1, NFR-6.1